Deep Neural Network Compression & CV
Mid-Evaluation Report - Group 4

Group Members

1. Abhinav Saxena (240037)
2. Eragandula Nehasri (240382)
3. Hrishita Singh (240460)
4. Raghav Dangayach (240826)
5. Samridhi Jain (240923)
6. Sarthak Patel (240940)
7. Shriya Suravarapu (241003)
8. T. Krishnapriya Reddy (241080)
9. Utkarsh Kashyap (241115)

Contents

1 Lecture 1: Image Basics and Fourier Transforms

1.1 Session Overview

Session 1, titled *Image Basics and Fourier Transforms (FFTs)*, introduced the fundamental mathematical and computational concepts required to understand how images are represented and processed in computer vision systems. The session focused on interpreting images as numerical matrices, converting them into suitable forms for computation, and analyzing visual information using both spatial and frequency domain techniques.

A major emphasis was placed on building intuition by connecting linear algebra, signal processing, and visual perception. Core topics covered included digital image representation, grayscale conversion, Fourier Transform theory, Fast Fourier Transform (FFT), interpretation of frequency components, magnitude–phase decomposition, and frequency-domain filtering.

1.2 Images as Mathematical Objects

1.2.1 Digital Image Representation

A digital image is fundamentally a matrix (or tensor) of numerical values.

Grayscale Image

$$I \in \mathbb{R}^{H \times W}$$

Color Image (RGB)

$$I \in \mathbb{R}^{H \times W \times 3}$$

Each pixel in a color image is represented as:

$$\text{Pixel} = (R, G, B), \quad R, G, B \in [0, 255]$$

For mathematical operations, images are converted from `uint8` to `float32`:

$$I_{\text{float}} = \frac{I}{255}$$

This conversion ensures numerical stability during convolution and Fourier operations.

1.3 Image Loading and Channel Ordering

Different libraries follow different channel conventions:

- **OpenCV**: BGR format
- **Matplotlib / NumPy**: RGB format

Hence, correct visualization requires:

$$I_{\text{RGB}} = \text{BGR} \rightarrow \text{RGB}$$

This step prevents incorrect color rendering and is critical for preprocessing pipelines.

1.4 Grayscale Conversion

Many computer vision algorithms operate on grayscale images due to reduced complexity and improved interpretability.

1.4.1 Weighted Grayscale Conversion

The luminance-based grayscale conversion is defined as:

$$I_{\text{gray}} = 0.299R + 0.587G + 0.114B$$

This weighting aligns with human visual perception, giving more importance to green and red channels.

1.4.2 Why Grayscale?

- Simplifies computation
- Improves edge detection
- Easier frequency analysis
- Reduces memory overhead

1.4.3 Frequency Domain Analysis

1.4.4 Fourier Transform Intuition

The Fourier Transform decomposes an image into a weighted sum of sinusoidal basis functions:

$$f(x, y) \xrightarrow{\mathcal{F}} F(u, v)$$

where:

- (x, y) denote spatial coordinates
- (u, v) denote frequency coordinates

2 Discrete Fourier Transform (DFT)

For a discrete image of size $M \times N$, the DFT is defined as:

$$F(u,v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y)\, e^{-j2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)}$$

2.0.1 Computational Complexity

- **DFT**: $\mathcal{O}(N^2)$
- **FFT**: $\mathcal{O}(N \log N)$

FFT enables real-time image processing and is indispensable in modern computer vision systems.

2.1 Frequency Interpretation

2.1.1 Low Frequencies

- Smooth regions
- Background illumination
- Gradients
- Large-scale structures

2.1.2 High Frequencies

- Edges
- Textures
- Noise
- Fine details

For visualization, frequency components are shifted:

$$F_{\text{shift}} = \texttt{fftshift}(F)$$

This operation centers low-frequency components in the spectrum.

2.2 Magnitude and Phase Spectrum

The Fourier Transform yields complex values:

$$F(u, v) = |F(u, v)|e^{j\angle F(u,v)}$$

2.2.1 Magnitude Spectrum

$$\text{Magnitude} = \log\left(|F(u, v)| + 1\right)$$

- Indicates strength of frequency components
- Bright center implies dominant low frequencies

2.2.2 Phase Spectrum

$$\text{Phase} = \angle F(u, v)$$

- Encodes structural and spatial information
- Preserves edges and object outlines

Key Insight *Phase carries image structure, while magnitude carries texture.*

2.3 Frequency Domain Filtering

2.3.1 Low-Pass Filter (LPF)

A Low-Pass Filter preserves low frequencies while removing high frequencies.

Effect

- Blurring
- Noise reduction
- Smooth transitions

Mask definition:

$$M_{\text{LPF}}(u,v) = \begin{cases} 1, & (u-u_0)^2 + (v-v_0)^2 \leq r^2 \\ 0, & \text{otherwise} \end{cases}$$

2.3.2 High-Pass Filter (HPF)

A High-Pass Filter preserves high frequencies while removing low frequencies:

$$M_{\text{HPF}} = 1 - M_{\text{LPF}}$$

Effect

- Edge detection
- Sharpening
- Detail extraction

2.4 Convolution in Frequency Domain

Using the Convolution Theorem:

$$f(x,y) * h(x,y) \mathcal{F} F(u,v) \cdot H(u,v)$$

2.4.1 Filtering Steps

1. Compute FFT
2. Shift frequencies
3. Apply mask
4. Inverse shift
5. Inverse FFT
6. Take absolute value

$$I_{\text{filtered}} = \left| \mathcal{F}^{-1}(F_{\text{shift}} \cdot M) \right|$$

2.5 Experimental Results

The experiments demonstrated:

- Blurred outputs from Low-Pass Filters
- Edge-enhanced outputs from High-Pass Filters
- Clear separation of frequency components
- Reconstruction of meaningful structures using phase alone

These results highlight the power of frequency-domain analysis in computer vision systems.

2.6 Key Takeaways

- Images are matrices governed by linear algebra
- Fourier Transform reveals hidden frequency structures
- FFT enables efficient large-scale image processing
- Phase is more critical than magnitude for structure
- Frequency filtering is foundational for denoising and edge detection

2.7 Conclusions and Reflections

This session provided a strong conceptual foundation for computer vision by unifying mathematics, signal processing, and visual intuition. Understanding images in the frequency domain reshapes how one approaches filtering, feature extraction, and later deep learning architectures.

The emphasis on experimentation reinforced the idea that computer vision is best learned by doing rather than merely observing results. This session set the groundwork for advanced topics such as convolutional neural networks, edge detectors, and image restoration techniques.

Lecture 2: Histograms

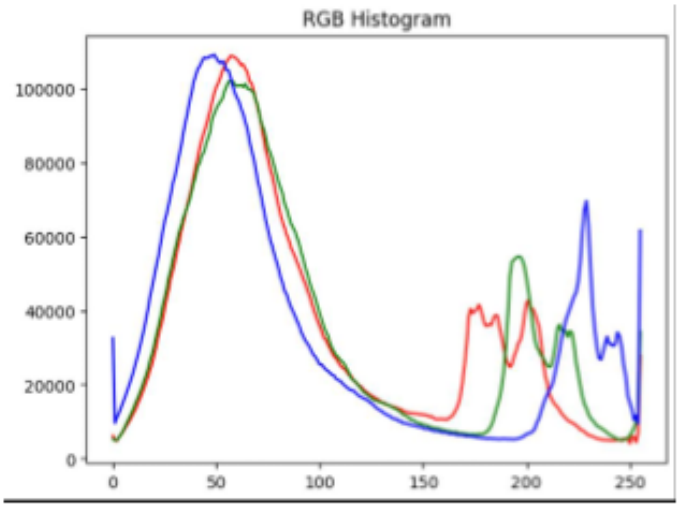
2.1 Histograms

A histogram is a plot which shows how many pixels exist at each intensity level (from 0 to 255). It represents the brightness distribution of an image.

- **Grayscale Histogram:** Plots intensity on the X-axis and pixel count on the Y-axis.
- **Inferences:** We can infer image properties by analyzing the histogram shape:
  - **Left-leaning:** Dark image (most pixels have low values).
  - **Centered:** Balanced exposure.
  - **Right-leaning:** Bright image (most pixels have high values).
  - **Narrow:** Low contrast (pixels clustered in a small intensity range).
  - **Wide:** High contrast (pixels spread across the full range).

RGB Histograms show the intensity distribution for red, green, and blue channels separately on a single diagram.

- **High R values:** Signals a warm image.
- **High B values:** Signals a cold image.
- **Unequal Peaks:** Signals color imbalance.




```
#Grayscale Histogram using Matplotlib
plt.hist(gray.ravel(), bins=256,range=(0,256))
plt.title("Grayscale Histogram")
plt.show()
```

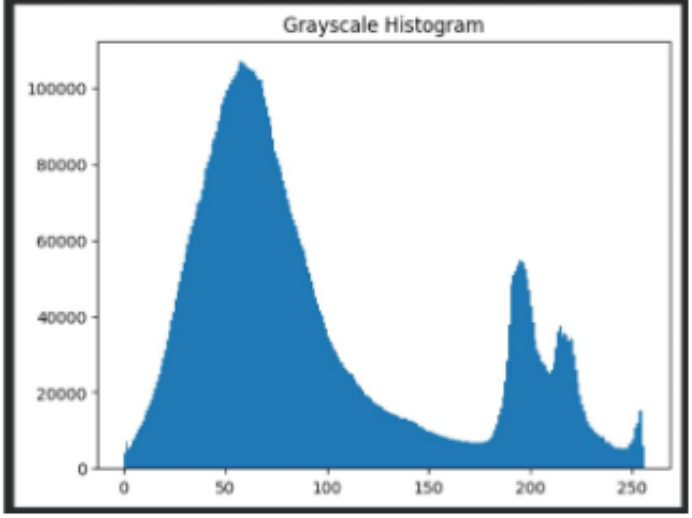


Figure 1: RGB and Grayscale Histograms

2.2 Color Spaces

HSV Color Space

- **Hue (H):** Represents the purest form of a color (0 to 180 in OpenCV).
- **Saturation (S):** Represents how vivid or pure a color is.
- **Value (V):** Represents the brightness or lightness of each pixel.

HSL Color Space

- Stands for Hue, Saturation, and Lightness.
- **Lightness (L):** Chooses the midpoint between black and white ($L = 0$ is Black, $L = 1$ is White, $L = 0.5$ is True Color).
- Saturation is strongest at $L = 0.5$ and weak at extremes.

2.3 Image Parameters

Contrast

- Refers to the difference in brightness between different parts of the image.
- **Increased Contrast:** Enhances the difference between light and dark areas, making the image more vivid, but may result in lost detail in shadows.

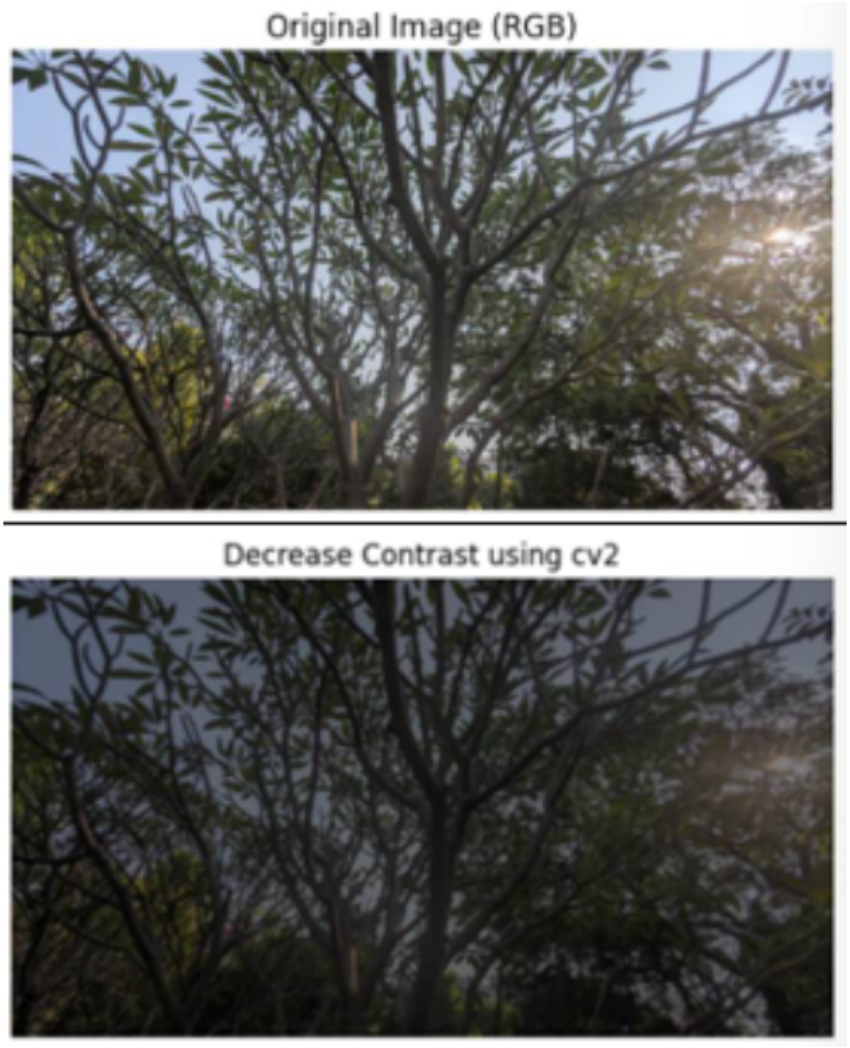


Figure 2: Comparison: Original vs. Decreased Contrast

Brightness

- A measure of the overall intensity or luminance of pixels.
- Increasing brightness adds a constant value β to pixel values:

$$\text{New} = \text{Old} + \beta \tag{1}$$

- This shifts the histogram to the right without increasing contrast.
- **Clipping:** If the image becomes too bright/dark, highlights turn pure white or shadows turn pure black.

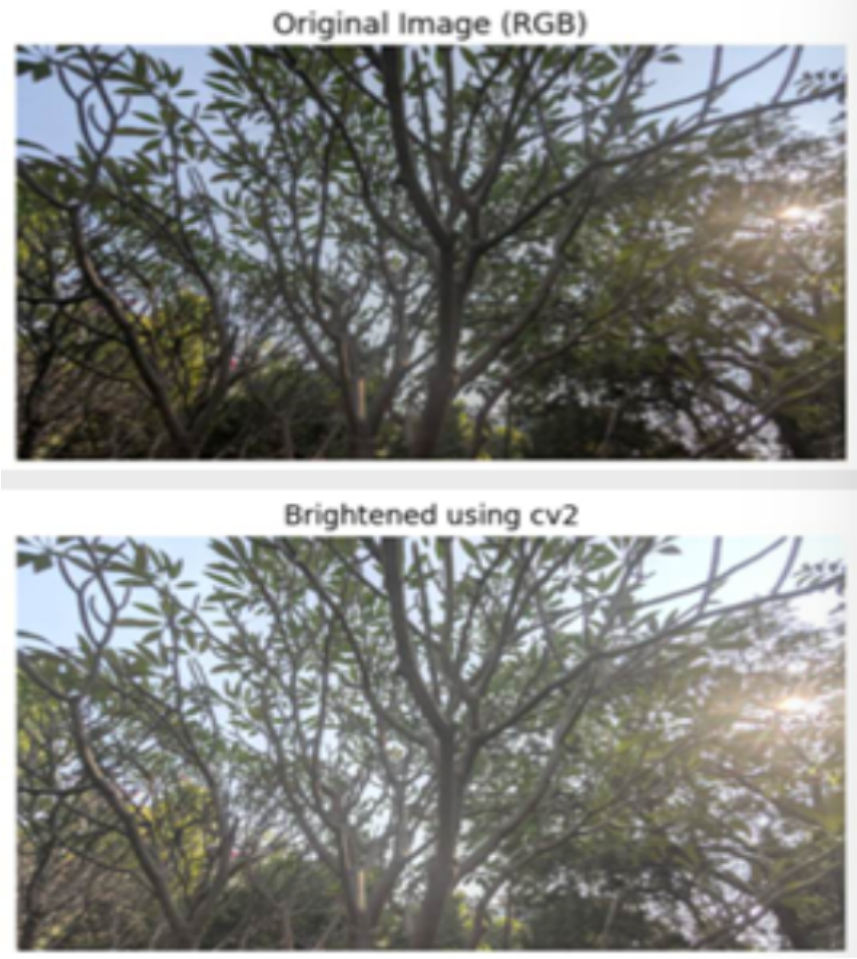


Figure 3: Comparison: Original vs. Brightened Image

Saturation & Hue

- **Saturation:** High saturation indicates vibrant colors. In HSV, increasing S makes color more pronounced, while $S = 0$ is grayscale.
- **Hue:** Manipulating hue is equivalent to rotating the color wheel (e.g., $+20°$ shifts reds to orange) while preserving saturation and brightness.

Figure 4: Comparison: Original vs. Increased Saturation

Figure 5: Hue Shift Example (120°)

Gamma Correction

- Gamma (γ) is a non-linear exponent used to match human perception, as eyes are more sensitive to dark tones than bright tones.
- Images often store values in a gamma-compressed form rather than linearly.

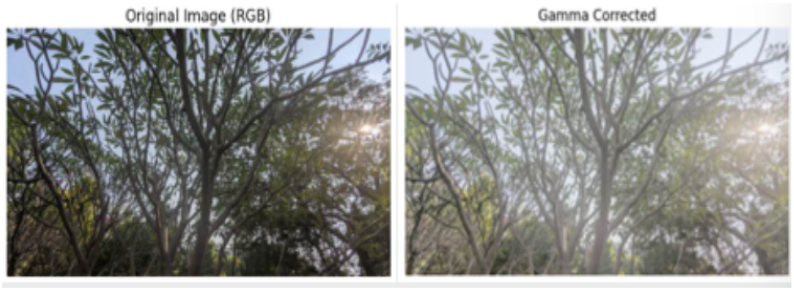


Figure 6: Comparison: Original vs. Gamma Corrected

2.4 White Balance and Vibrance

White Balance

- Corrects color cast so whites appear white by neutralizing dominant colors (e.g., yellow in indoor lighting).
- Mathematical approaches include the Gray World Assumption and White Patch Method.

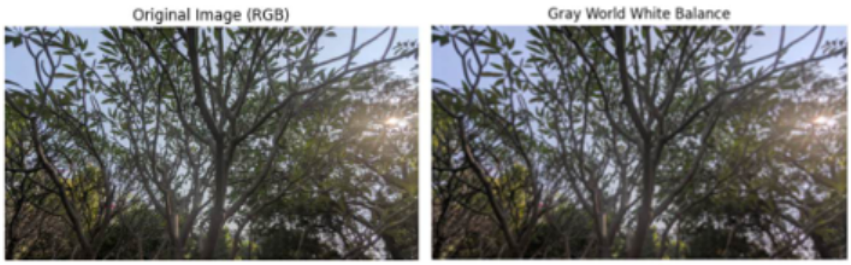


Figure 7: Comparison: Original vs. White Balanced (Gray World)

Vibrance vs. Saturation

- **Saturation:** Boosts all colors equally, which can affect skin tones and look unnatural.
- **Vibrance:** Selective saturation that boosts only low-saturation pixels, protecting skin tones.
- **Implementation:** Vibrance applies boosting mainly when saturation (S) is low:

$$S' = S + \alpha \cdot (1 - S) \cdot S \tag{2}$$

Lecture 3: Computer Vision - Convolution, Gradients and Edge Detection

3.1 Convolutions and Kernels

Convolution was introduced as the fundamental operation behind most image filtering techniques. A kernel (matrix) is slid across the image and each pixel value is replaced by a weighted sum of its neighbours. Depending on the kernel design, convolution can:

- Blur images
- Sharpen details
- Extract edges
- Detect directions
- Smooth noise
- Highlight textures

```
def convolve(gray_img,kernel):
  gray_img=gray_img.astype(np.float64)
  kh,kw=kernel.shape
  pad_h=kh//2
  pad_w=kw//2
  padded_img=np.pad(gray_img,((pad_h,pad_h),(pad_w,pad_w)),mode='reflect')
  H,W=gray_img.shape
  output=np.zeros((H,W))
  for i in range(H):
      for j in range(W):
          region=padded_img[i:i+kh,j:j+kw]
          output[i,j]=np.sum(region*kernel)
  return output
convolved_img=convolve(gray_img,kernel)
plt.title("Original")
plt.imshow(rgb_img)
plt.axis("off")
plt.show()
plt.imshow(convolved_img,cmap='gray')
plt.title("Convolved image gray")
plt.axis("off")
plt.show()
```

Figure 8: Convolve function using NumPy as done in the assignment

3.2 Blurring Techniques

Two primary blurring methods were discussed:

- **Average Blur:** Assigns equal weight to all neighbouring pixels, this reduces noise, removes fine details, and makes the image smooth.

$$\text{Kernel} = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

- **Gaussian Blur:** Assigns the highest weight to the centre pixel with smoothing decreasing weights outward. This reduces noise and preserves edges better than average blurring.

$$\text{Kernel} = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

```
avg_blur_cv = cv2.blur(gray, (3,3))
gaussian_cv = cv2.GaussianBlur(gray, (3,3), 0)
```

Figure 9: Code snippets

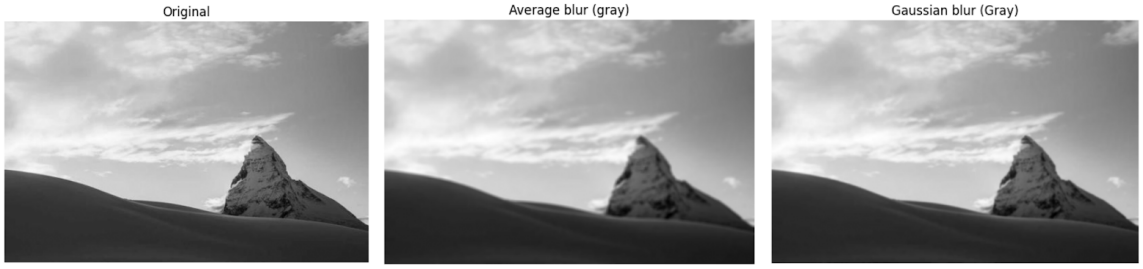


Figure 10: Blurring Techniques

3.3 Image Gradients

Gradients tell us how fast the pixel intensity changes and in which direction. Key observations include:

- Small gradients indicate smooth regions.
- Large gradients indicate sharp transitions.
- Maximum gradient magnitude occurs at edges.

The magnitude and direction are computed as follows:

- **Magnitude:** $S = ||\nabla I|| = \sqrt{(\frac{\partial I}{\partial x})^2 + (\frac{\partial I}{\partial y})^2}$
- **Direction:** $\theta = \tan^{-1}\left(\frac{\partial I/\partial y}{\partial I/\partial x}\right)$

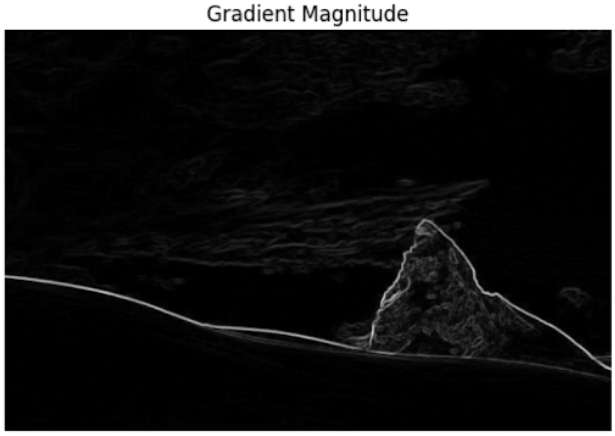


Figure 11: Image Gradients

3.4 Edge Detection

Sobel Operator: Involves convolution with x and y Sobel kernels, computing gradient magnitude, and optional thresholding to extract edges.

Canny Edge Detector: Produces thin, noise-resistant edges through a multi-step process:

1. Gaussian blurring
2. Sobel gradient computation
3. Laplacian operation
4. Non-maximum suppression
5. Double thresholding
6. Edge tracking by hysteresis

```
canny = cv2.Canny(gray, 100, 200)
plt.figure(figsize=(6,6))
plt.imshow(canny, cmap='gray')
plt.title("Grayscale Canny Edge Detection using cv2")
plt.axis("off")
plt.show()
```

Figure 12: Code snippet

3.5 Sharpening Methods

- **Laplacian Sharpening:** Uses second-order derivatives to highlight finer details. The sharpened image is obtained by subtracting the Laplacian from the original image.
- **Unsharp masking:** Blurs the image, computes a detail mask by subtracting the blur from the original, and adds a scaled version of this mask back.

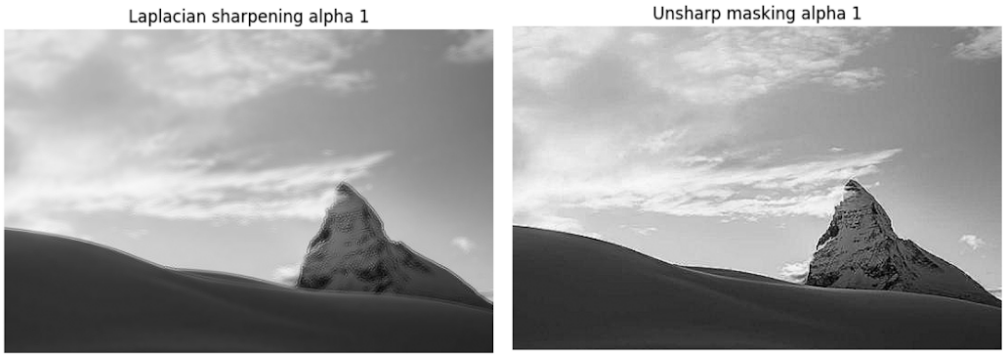


Figure 13: Sharpening Methods

3.6 Frequency Domain Representation

A key takeaway was the equivalence between convolution and frequency-domain filtering:

- Gaussian blur acts as a **low-pass filter**, attenuating high frequencies.
- Sharpening acts as a **high-pass filter**, amplifying high frequencies.

Lecture 4: Feature Detection and Shape Detection in Computer Vision

Introduction

This session focused on fundamental image processing techniques used in computer vision, including feature detection methods, color spaces, and shape detection using the Hough Transform. These concepts form the foundation for understanding how computers analyze and interpret visual information from images.

3 Corner Detection

Corners in an image can be identified by analyzing small patches around pixels.

- In a flat region, shifting the patch in any direction produces little or no change in intensity.
- In an edge region, intensity changes occur only in the direction perpendicular to the edge.
- In a corner region, intensity changes occur in all directions.

This observation motivates the need for a function that can measure intensity changes under small shifts. For this purpose, image gradients in the horizontal and vertical directions are computed.

3.1 Image Gradients

The image gradients along the x and y directions are defined as:

$$I_x = \frac{\partial I}{\partial x}, \qquad I_y = \frac{\partial I}{\partial y} \tag{3}$$

These gradients are commonly computed using Sobel filters.

4 Hough Transform for Line Detection

Unlike edge detection methods such as Sobel or Canny filters, the Hough Transform incorporates prior knowledge about object shape and structure to detect geometric shapes.

4.1 Duality Principle

The Hough Transform relies on the concept of duality:

- A point (x, y) in image space corresponds to a line in parameter space.
- A line in image space corresponds to a point in parameter space.

4.2 Voting Mechanism

Each edge point in the image votes for all possible lines that could pass through it. These votes are accumulated in an accumulator array, and peaks in this array indicate the presence of lines.

4.3 Polar Coordinate Representation

To avoid issues with infinite slopes, the line equation is expressed in polar coordinates:

$$\rho = x \cos\theta + y \sin\theta \tag{4}$$

or equivalently,

$$x \cos\theta + y \sin\theta - \rho = 0 \tag{5}$$

Here, ρ is the perpendicular distance from the origin to the line, and θ is the angle between the normal and the x-axis.

5 Detecting Circles and Ellipses

5.1 Circle Detection

A circle in an image is described by the equation:

$$(x - a)^2 + (y - b)^2 = r^2 \tag{6}$$

If the radius r is known, the parameter space is two-dimensional (a, b). If the radius is unknown, the parameter space becomes three-dimensional (a, b, r), increasing computational complexity.

5.2 Ellipse Detection

Ellipse detection requires a five-dimensional parameter space representing the center, major and minor axes, and orientation. This makes ellipse detection computationally expensive using the standard Hough Transform.

6 Generalized Hough Transform

The Generalized Hough Transform is used for detecting complex shapes that cannot be represented using simple parametric equations.

6.1 R-table

An R-table is constructed by mapping edge orientations θ to vectors $\mathbf{r}$ pointing toward a reference point:

$$\theta \rightarrow \mathbf{r} \tag{7}$$

6.2 Handling Transformations

If the object undergoes scaling s or rotation θ, the accumulator space must be extended to include these parameters:

$$(x, y, s, \theta) \tag{8}$$

While robust to noise, occlusion, and disconnected edges, the Generalized Hough Transform is memory-intensive and computationally expensive.

7 Assignment Insights

The assignment associated with this lecture requires implementing the discussed algorithms from scratch using `numpy`. High-level library functions are avoided to ensure a strong understanding of the underlying mathematical concepts.

7.1 Part A: Gradient Analysis and Scatter Plots

The objective is to analyze the relationship between horizontal and vertical gradients.

- Gradients I_x and I_y are computed using Sobel kernels.

- Three types of image regions are analyzed: flat, edge, and corner.
- Scatter plots of I_y versus I_x are generated.

In flat regions, points cluster near the origin. Edge regions form linear structures, while corner regions show a wide spread of points.

7.2 Part B: Harris Corner Detection and R-maps

The Harris corner detector quantifies the "cornerness" of a pixel using the second moment matrix:

$$M = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix} = \begin{bmatrix} a & c \\ c & b \end{bmatrix} \tag{9}$$

The Harris response is given by:

$$R = \det(M) - k(\text{trace}(M))^2 \tag{10}$$

or equivalently,

$$R = (ab - c^2) - k(a + b)^2 \tag{11}$$

An R-map is generated where pixel intensity corresponds to the Harris response value, making corners appear as bright peaks.

7.3 Part C: Hough Transform for Line Detection

The goal is to detect lines by transforming from image space to parameter space.

- Lines are detected using both slope-intercept (m, c) and polar (ρ, θ) representations.
- A Canny edge map is generated.
- Each edge pixel votes in the accumulator array.
- Thresholding is applied to identify dominant lines.

7.4 Part D: Generalized Hough Concepts (Bonus)

In this part, gradient information and reference stencils are used to isolate and move objects.

A specific object is identified using contours and gradients, and a reference point is used (similar to the R-table method) to overlay the object onto a new location in a different image.

Conclusion

This lecture provided a comprehensive understanding of feature detection and shape detection techniques in computer vision. While methods like the Hough Transform are powerful and robust, their computational cost increases significantly with object complexity and parameter dimensionality.

Lecture 5: Machine Learning Fundamentals-Regression, Classification and Clustering

1. Types of Learning Paradigms

1.1 Supervised Learning

In supervised learning, the dataset consists of input–output p... function that maps input values to corresponding output values.

Regression: Output yyy is continuous (e.g., predicting house prices).

Classification: Output yyy belongs to discrete classes (e.g., spam vs non-spam).

1.2 Unsupervised Learning

In unsupervised learning, only inputs xxx are available, and there are no labels.

The objective is to discover patterns or structure in the data.

A common example is clustering, such as the K-means algorithm.

2. Maximum Likelihood Estimation (MLE)

2.1 Linear Regression Model

In linear regression, the predicted output is modelled as:

$$\hat{y} = h_w(x) = w^T x \tag{12}$$

where:

w is the D-dimensional vector of model weights (parameters).

x is the D-dimensional feature vector.

2.2 MLE for Regression

Given a dataset $\{(x_i, y_i)\}_{i=1}^n$, the likelihood function is:

$$\mathcal{L}(w) = \prod_{i=1}^{n} P(y_i \mid x_i, w) \tag{13}$$

Taking logarithm:

$$\log \mathcal{L}(w) = -\frac{1}{2\sigma^2} \sum_{i=1}^{n} \left(y_i - w^T x_i\right)^2 + \text{constant} \tag{14}$$

Maximizing log-likelihood is equivalent to minimizing Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} \left(y_i - w^T x_i\right)^2 \tag{15}$$

Thus,

$$\arg\max_w \log \mathcal{L}(w) = \arg\min_w \text{MSE} \tag{16}$$

2.3 MLE for Classification

For binary classification, let:

$P_i = P(y_i = 1)$

$1 - P_i = P(y_i = 0)$

The likelihood is:

$$\mathcal{L} = \prod_{i \in y=1} P_i \prod_{i \in y=0} (1 - P_i) \tag{17}$$

Taking log:

$$\log \mathcal{L} = \sum_{i \in y=1} \log P_i \; + \sum_{i \in y=0} \log(1 - P_i) \tag{18}$$

This forms the basis of logistic regression and binary cross-entropy loss.

3. Is Linear regression really linear?

Although linear regression is linear in parameters, it can mo...ionships through feature mapping. Example: Polynomial regression

Original model:

$$y = ax^3 + bx^2 + cx + d \tag{19}$$

Feature transformation:

$$\phi(x) = [1, x, x^2, x^3] \tag{20}$$

New model:

$$y = w^T \phi(x) \tag{21}$$

Thus, linear regression can model non-linear functions while remaining linear in parameters.

4. Overfitting and Regularisation

When the number of features exceeds the number of data points... overfit, achieving zero training error but poor generalization.

To address this, regularisation is introduced:

$$\text{Loss} = \text{MSE} + \lambda \sum_j w_j^2 \tag{22}$$

This is known as L2 regularization (Ridge Regression) and penalizes large weights.

5. K-Means ++

The problem with Standard K-Means is random initialization that can place centroids too close, leading to poor clustering.

Steps for K-Means Initialisation:

1. Select the first centroid uniformly at random from the dataset.
2. Select the next centroid with probability:

$$P(x_i) \propto \|x_i - \mu\|^2 \tag{23}$$

3. Repeat until k centroids are chosen.

This ensures centroids are well separated, improving convergence and clustering quality.

Lecture 6: Model Generalization and Neural Networks

Abstract

This report summarizes the theoretical foundations and practical implications of model generalization in machine learning, covering underfitting, overfitting, regularization, feature scaling, and neural network fundamentals. The lecture bridges classical regression models and modern neural architectures, emphasizing robustness, optimization, and generalization.

1. Introduction

1.1 Session Overview

Lecture 6 focuses on the critical concept of generalization—the ability of a machine learning model to perform accurately on new, unseen data rather than just the training set. This session builds upon previous discussions regarding linear regression, loss functions, and Mean Squared Error (MSE).

1.2 Learning Objectives

- Distinguish between underfitting and overfitting.
- Implement regularization techniques to enhance model robustness.
- Understand the architecture of Neural Networks and the mechanics of Backpropagation.

2. Underfitting

2.1 Concept and Intuition

Underfitting occurs when a model is too simple to capture the underlying structure or patterns within the data. It essentially fails to learn the trend of the training data.

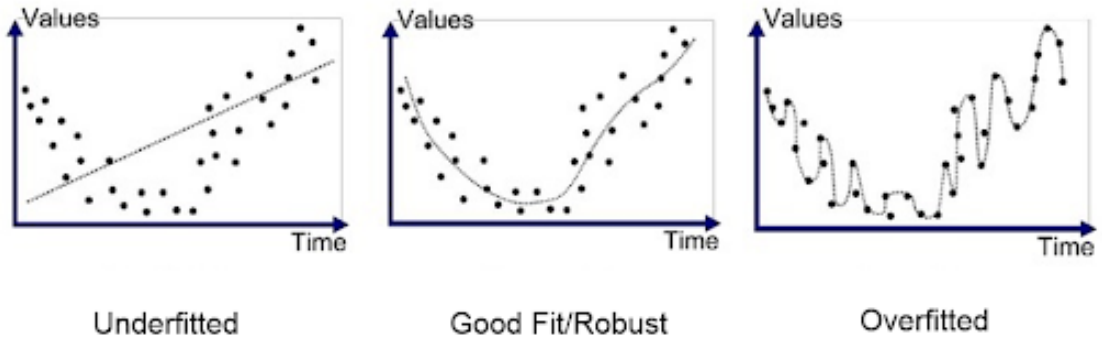


Figure 14: Comparison of underfitting, appropriate model fitting, and overfitting, illustrating how model complexity affects the ability to capture underlying data trends.

2.2 Mathematical Perspective

A typical example is attempting to fit a linear model to non-linear data.

$$f(x) = wx + b$$

Such models lack the complexity (degrees of freedom) required to represent non-linear relationships, leading to high error rates.

2.3 Practical Implications

- High Bias: The model makes strong, incorrect assumptions about the data.
- Poor Performance: High error is observed on both training and test datasets.

3. Overfitting

3.1 Concept and Intuition

Overfitting happens when a model is excessively complex and begins to "memorize" the training data, including its noise and outliers, rather than learning the general pattern. While the model fits the training data perfectly, it fails to generalize.

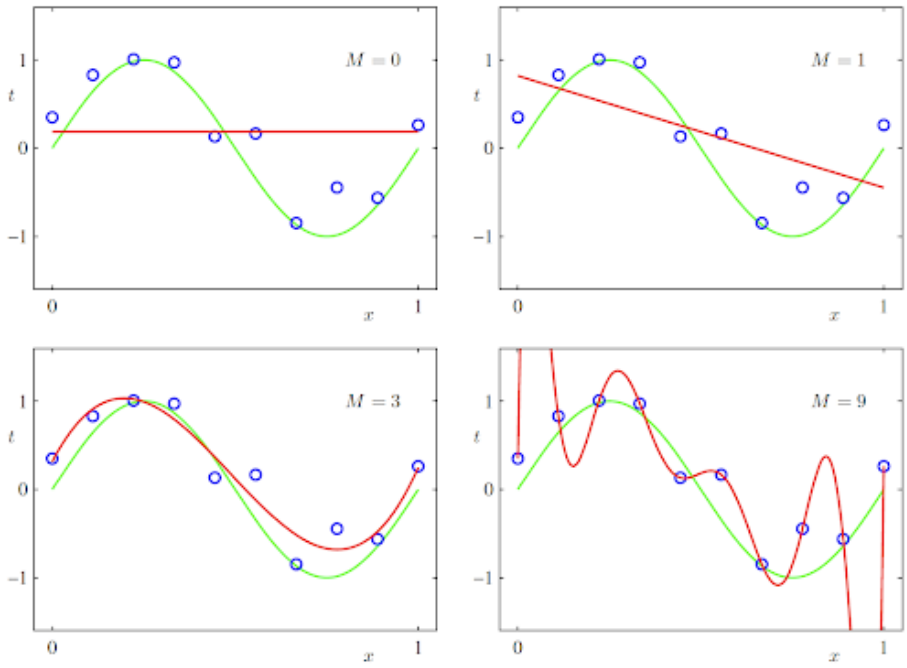


Figure 15: Illustration of overfitting using polynomial regression, showing how increasing model complexity (polynomial degree) leads from underfitting to overfitting of the training data.

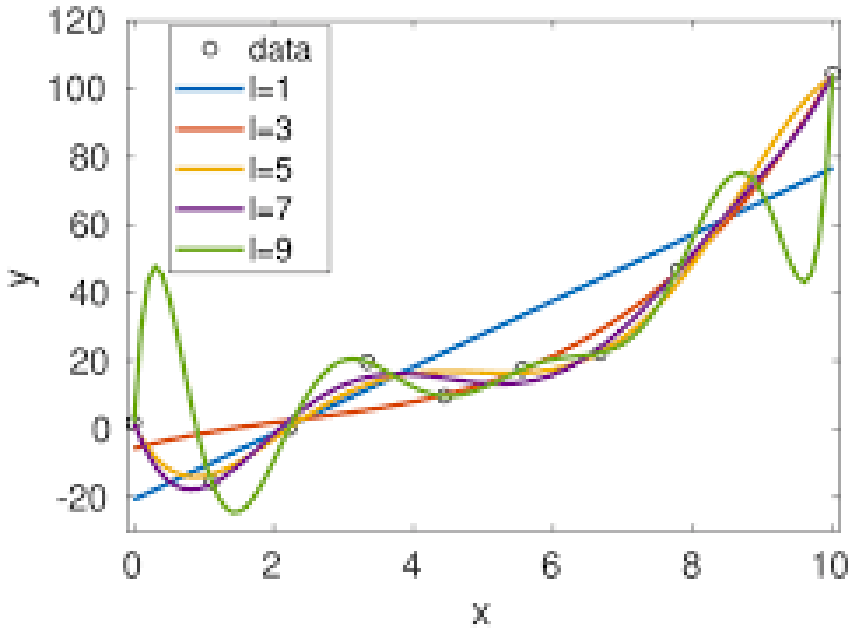


Figure 16: Effect of increasing model complexity on data fitting, highlighting how overly complex models capture noise and result in unstable predictions.

3.2 Illustration via Polynomial Models

Overfitting is often illustrated using high-degree polynomials. For instance, a model with many features (e.g., x^1, x^2, x^3, x^4, x^5) can pass through every single training point.

$$f(x) = w_1x + w_2x^2 + w_3x^3 + w_4x^4 + w_5x^5 + b$$

3.3 Training vs. Test Error Behavior

In an overfit model:

- $MSE_{train} \approx 0$
- MSE is significantly large.

4. Bias-Variance Tradeoff

4.1 Definition

- Bias: Error due to overly simplistic assumptions (underfitting).
- Variance: Error due to excessive sensitivity to small fluctuations in the training set (overfitting).

4.2 Tradeoff Explanation

As model complexity increases, bias decreases, but variance increases. The goal is to find the "sweet spot" where the total error (bias + variance) is minimized to achieve optimal generalization.

4.3 Graphical Interpretation

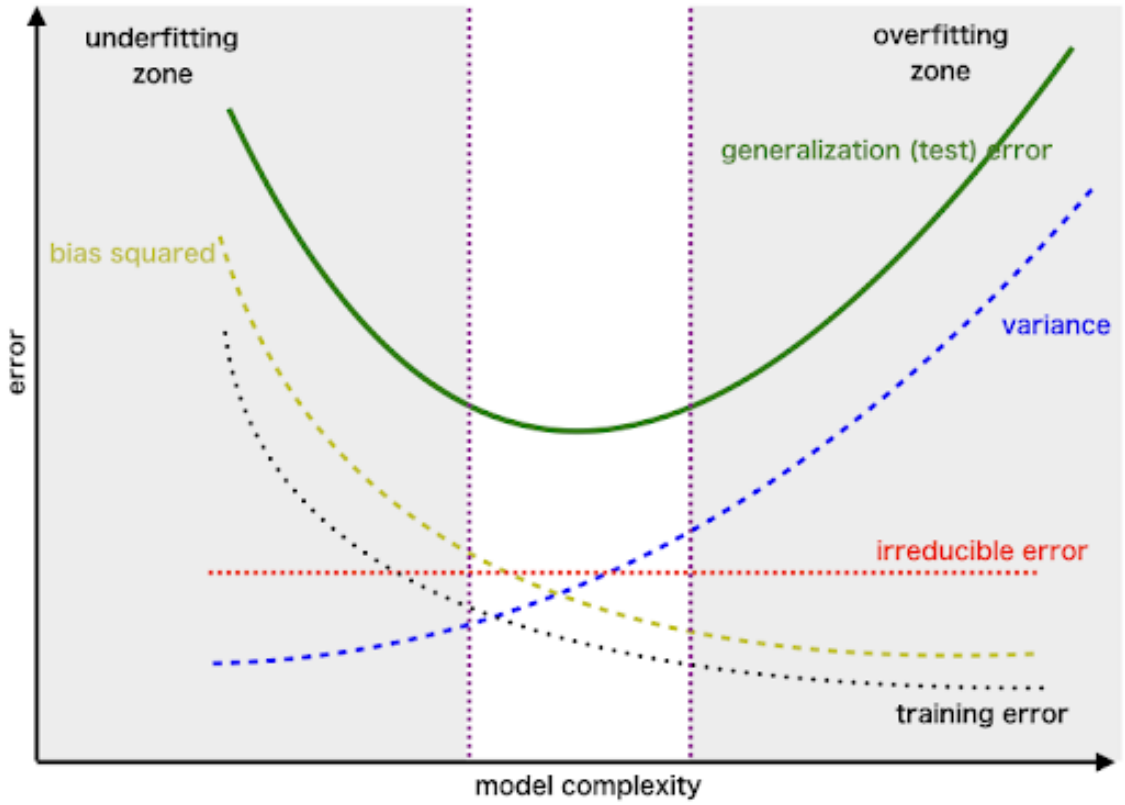


Figure 17: Graphical illustration of the bias–variance tradeoff, showing training error, test (generalization) error, bias, variance, and irreducible error as functions of model complexity. The optimal model lies at the minimum of test error.

5. Regularization Techniques

5.1 Motivation

Regularization is used to prevent overfitting by penalizing large weights, thereby constraining model complexity.

5.2 L2 Regularization (Ridge)

This technique adds a penalty term to the Mean Squared Error (MSE) loss function based on the magnitude of the weights.

$$Loss = MSE + \lambda \sum |w_i|^2$$

5.3 Effect of λ on Model Behavior

- Small λ: The model remains flexible and may still overfit.
- Large λ: The penalty for large weights is high, leading to a smoother, simpler model with reduced variance (weight shrinkage).

6. Feature Scaling

6.1 Need for Scaling

When features have vastly different ranges (e.g., house area in thousands vs. number of rooms in units), optimization becomes difficult and slow.

6.2 Standardization (Z-score Scaling)

Scaling ensures features are comparable in size, typically centering them around a mean (μ) of 0 with a standard deviation (σ) of 1.

$$x_{new} = \frac{x_{old} - \mu}{\sigma}$$

6.3 Impact on Optimization

- Faster Convergence: Gradient Descent reaches the minimum more quickly.
- Stable Gradients: Prevents weights from fluctuating rapidly due to scale imbalances.

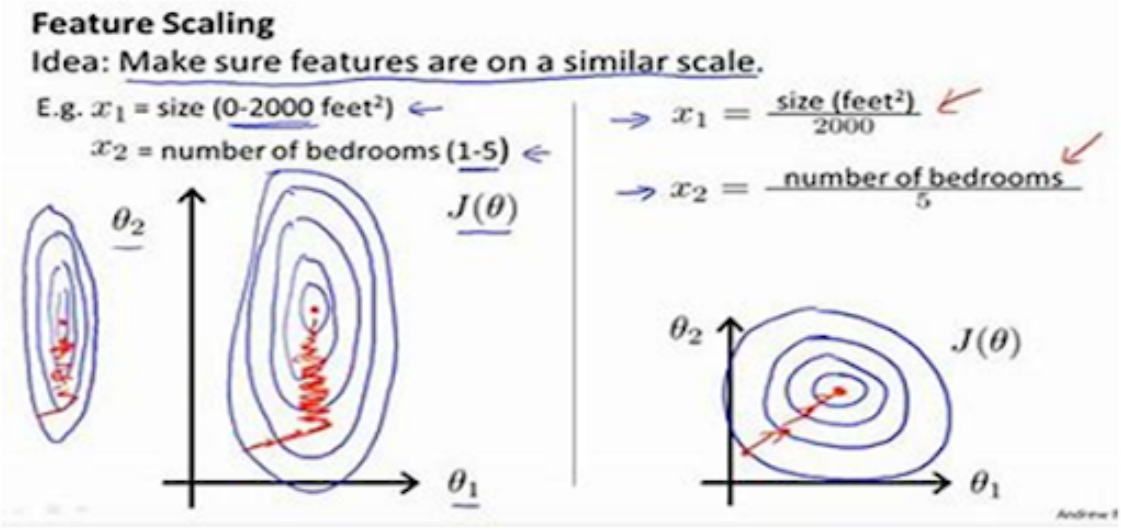


Figure 18: Effect of feature scaling on gradient descent optimization, illustrating faster and more stable convergence when input features are properly normalized.

7. Multi-Layer Perceptron (MLP)

7.1 Architecture

An MLP consists of several layers of neurons:

- Input Layer: Receives the feature vector.
- Hidden Layers: Intermediate layers where computations occur.
- Output Layer: Produces the final prediction.

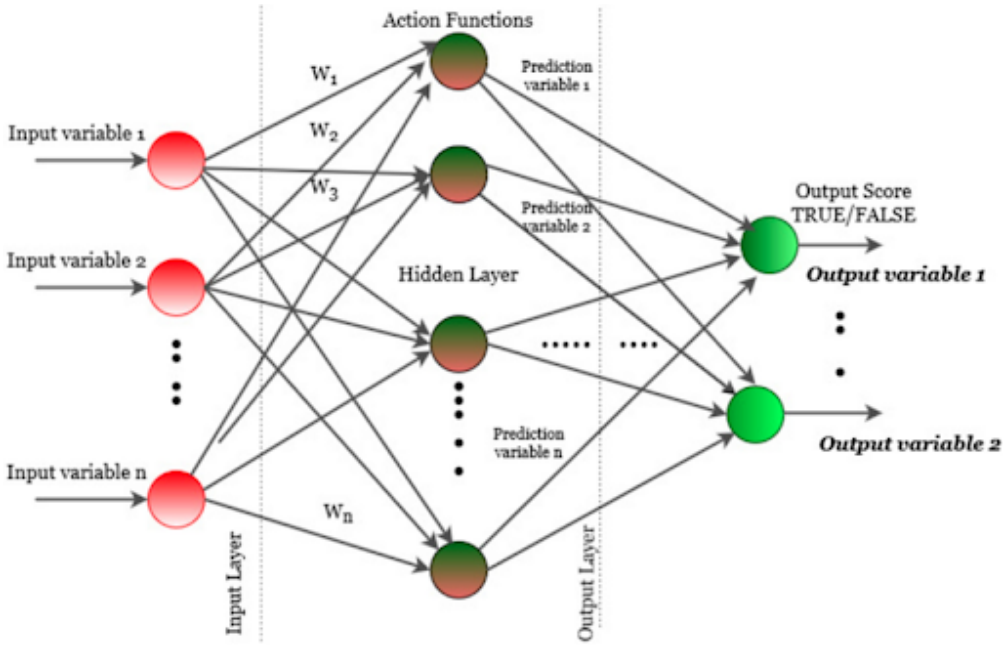


Figure 19: Architecture of a Multi-Layer Perceptron (MLP) showing input, hidden, and output layers with weighted connections and activation functions.

7.2 Mathematical Formulation and Nonlinearity

Without non-linear activation functions, a multi-layer network would just be a single linear transformation. The output of a hidden layer is typically:

$$h = ReLU(Wx + b)$$

8. Activation Functions and Universal Approximation

8.1 ReLU Activation

The Rectified Linear Unit (ReLU) is defined as:

$$ReLU(x) = \max(0, x)$$

8.2 Piecewise Linear Approximation

Summing multiple ReLU functions allows the network to create a piecewise linear approximation of any complex, non-linear function.

$$y = \sum_{i}^{K} ReLU(w_i x + b_i)$$

8.3 Universal Approximation Theorem

This theorem states that a neural network with even a single hidden layer and a non-linear activation function can approximate any continuous function to a desired degree of accuracy.

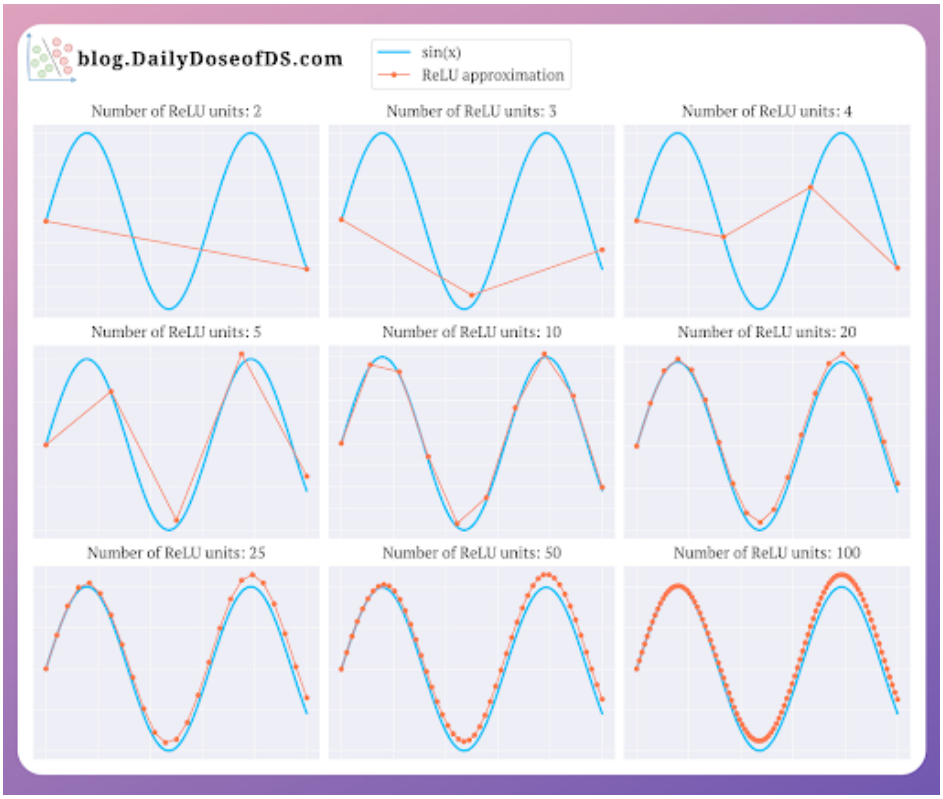


Figure 20: ReLU-based neural network approximating a continuous function with increasing hidden units.

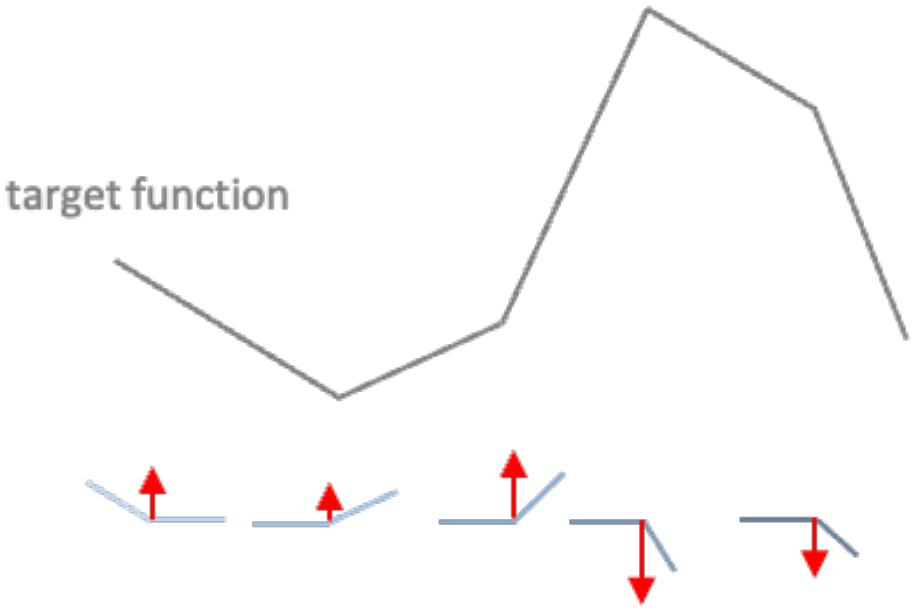


Figure 21: Conceptual illustration of the Universal Approximation Theorem, demonstrating how piecewise linear functions formed by ReLU neurons can approximate an arbitrary continuous target function.

9. Backpropagation and Optimization

9.1 Loss Function

The network's performance is measured using a loss function, such as the sum of squared errors:

$$L = \sum (y - \hat{y})^2$$

9.2 Gradient Computation and Update

Backpropagation uses the Chain Rule to calculate the gradient of the loss function with respect to each weight in the network.

$$\frac{\partial l}{\partial w} = \frac{\partial l}{\partial f} \cdot \frac{\partial f}{\partial w}$$

Weights are then updated to minimize the loss.

9.3 Weight Update Mechanism

$$w \leftarrow w - \eta \frac{\partial l}{\partial w}$$

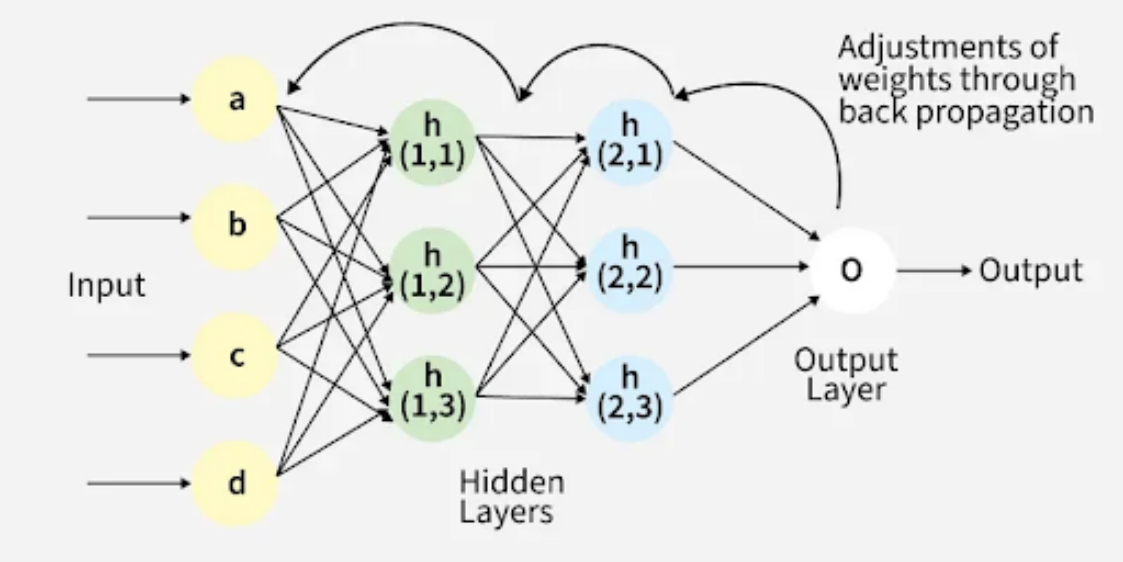


Figure 22: Weight update mechanism in a multi-layer neural network using backpropagation.

10. Key Insights and Learnings

- Generalization is the ultimate goal, balancing the tradeoff between bias and variance.
- Regularization (λ) is a powerful tool to control model complexity and prevent overfitting.
- Non-linearity (via activations like ReLU) is what gives neural networks their "power of depth," allowing them to model complex data.
- Backpropagation provides an efficient mathematical framework for optimizing large-scale neural architectures.

11. Conclusion

Lecture 6 successfully bridges classical statistical learning (regression, bias-variance) with modern deep learning (MLPs, backpropagation). Understanding how to scale features and regularize models is essential for building robust machine learning systems that can generalize effectively to real-world scenarios.

Lecture 7: Computer Vision Data Processing Pipeline

Introduction

This session describes a classical computer vision preprocessing pipeline used to enrich raw RGB images with structured representations and handcrafted features prior to model training.

7.1 Problem Setup and Dataset

We use the Fruits-360 dataset, consisting of RGB images with class labels. The raw image representation is augmented with processed images and linear features.

The final input formulation is:

$$\text{RGB Image} + \text{LBP Image} + \text{Canny Edge Image} + \text{Linear Features} \rightarrow \text{Fruit Label} \tag{24}$$

7.2 Color Feature Extraction (HSV Space)

RGB images are converted to HSV color space to decouple color from illumination. Six color features are extracted:

- Mean and standard deviation of Hue
- Mean and standard deviation of Saturation
- Mean and standard deviation of Value

These features capture dominant color, color variation, and brightness.

7.3 Shape Feature Extraction from Binary Masks

Shape features are extracted from the largest contour obtained after background segmentation. Normalized geometric descriptors are used.

7.4 Geometric Shape Features

Area Ratio

$$\frac{\text{Contour Area}}{\text{Image Area}} \tag{25}$$

Aspect Ratio

$$\frac{w}{h} \tag{26}$$

Solidity

$$\frac{\text{Contour Area}}{\text{Convex Hull Area}} \tag{27}$$

Circularity

$$\frac{4\pi A}{P^2} \tag{28}$$

where A is the contour area and P is the contour perimeter.

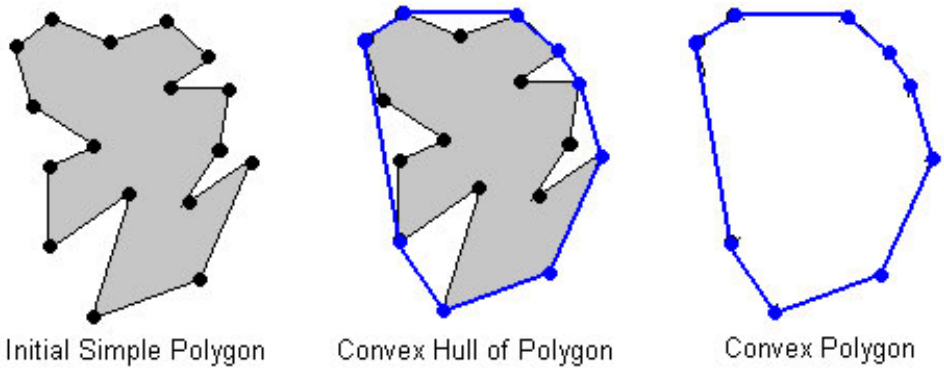


Figure 23: Convex hull formation from an irregular polygon.

7.5 Hu Moments

To capture higher-level shape characteristics, Hu moments are used. Only the first two Hu moments are retained in log scale:

- ϕ_1: overall shape spread
- ϕ_2: shape variance

Higher-order moments are omitted due to instability and noise sensitivity.

7.6 Texture Features Using Local Binary Patterns (LBP)

Texture is extracted using Local Binary Patterns. Each neighboring pixel is compared with the center pixel to generate an 8-bit binary code arranged clockwise.

The LBP operator is defined as:

$$\text{LBP} = \sum_{p=0}^{P-1} s(g_p - g_c) \cdot 2^p \tag{29}$$

$$s(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

Implementation

```
def get_lbp_image(gray):
    lbp = np.zeros_like(gray, dtype=np.uint8)
    neighbors = [(-1,-1), (-1,0), (-1,1),
                 (0,1), (1,1), (1,0),
                 (1,-1), (0,-1)]
    for idx, (dy, dx) in enumerate(neighbors):
        shifted = np.roll(np.roll(gray, dy, axis=0), dx, axis=1)
        lbp |= ((shifted >= gray) << (7 - idx))
    return lbp
```

A normalized histogram of LBP codes is used as the texture feature vector.

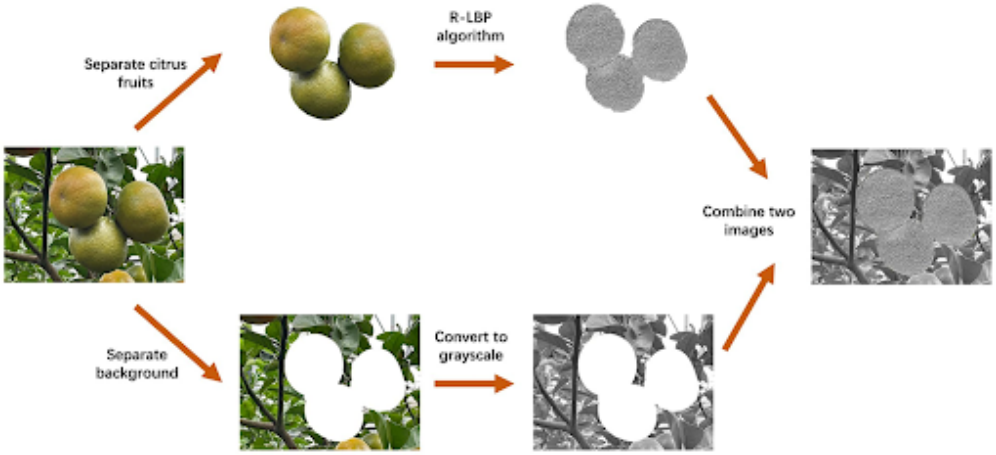


Figure 24: Output of LBP Algorithm

7.7 Edge Detection Using Canny Operator

Canny edge detection is applied to obtain a structural representation of object boundaries:

```
edges = cv2.Canny(gray, 100, 200)
```

Edge images are retained as an auxiliary image modality.

7.8 Final Feature Composition

The final representation consists of:

- 6 color features (HSV moments)
- 6 shape features (area ratio, aspect ratio, solidity, circularity, ϕ_1, ϕ_2)
- 2 processed images (LBP and Canny)

These components are returned together through a custom dataloader.

Summary

This session establishes a compact and interpretable computer vision preprocessing pipeline combining color, shape, texture, and edge-based representations for downstream classification.

Lecture 8: Deep Learning and Model Compression

8.1 Limitations of Standard Architectures

- **MLP (Multilayer Perceptron):** While MLPs can approximate any function (Universal Approximation Theorem) using piecewise linear functions like ReLU, they are inefficient for high-dimensional data like images.
- **Computational Cost:** A standard input of 1000 pixels requires many weights, making the model computationally heavy and difficult to deploy.
- **Transition to CNNs:** Convolutional Neural Networks (CNNs) are introduced to solve this. They rely on **Translational Invariance** (recognizing features anywhere in the image) and **Max Pooling** steps to reduce spatial dimensions and parameter count.

8.2 Deep Network Compression Techniques

1. Pruning (Sparsity)

- **Concept:** Removing unnecessary connections to create a sparse network.
- **Methodology:** First analyze the weight distribution, then select a threshold. Any weight below this threshold is pruned (set to 0).
- **Outcome:** Reduces the number of active parameters, saving memory.

2. Weight Sharing & Quantization

- **Storage Problem:** Storing weights as 32-bit floats is expensive.
- **Solution (K-Means Clustering):** Weights are grouped into clusters using K-Means. Only the centroid (mean value) of each cluster is stored in a "Codebook" or Hash Table.
- **Compression Mechanism:** The network stores a small index (e.g., 4 bits or 8 bits) pointing to the centroid in the codebook, rather than the full 32-bit value.

The compression relationship is given by:

$$32 \text{ bits} \rightarrow \text{Index bits} + \text{Codebook overhead} \tag{30}$$

8.3 Training & Optimization

- **Fine-tuning Compressed Models:** The lecture addressed how to update weights when they are shared or clustered.
- **Gradient Aggregation:** Gradients are calculated for all connections belonging to a specific cluster. These gradients are summed to update the single centroid value.
- **Mathematical Note:** Although the cluster center represents a mean, the derivative calculation simplifies by absorbing the division factor $(1/N)$ into the learning rate. The update effectively relies on the summation of gradients for that cluster:

$$\Delta w_{\text{centroid}} = \eta \sum_{i \in \text{cluster}} \frac{\partial \mathcal{L}}{\partial w_i} \tag{31}$$

Lecture 9: The Core of PyTorch—Tensors, Data and Gradients

9.1 PyTorch vs TensorFlow

9.2 How to teach a Machine to learn?

1. **Forward Pass:** Predict
2. **Compute loss:** Measure the error
3. **Backward pass:** Calculate $\frac{\partial \text{loss}}{\partial w}$
4. **Update weights:** $w = w - \eta \frac{\partial \text{loss}}{\partial w}$

Feature	PyTorch (The Winner)	TensorFlow (The Challenger)
Philosophy	"Python-first" (Dynamic)	"Graph-first" (Static/Hybrid)
Research Adoption	∼92% of 2024/25 papers	∼8% and shrinking
GenAI Support	Default for LLMs & Diffusers	Secondary support
Debugging	Easy (Standard Python tools)	Difficult (Complex stack traces)
Leading Edge	High-performance (PyTorch 2.x)	Mature but slower to iterate

Table 1: Comparison: At a Glance

9.3 Tensor

It is a multidimensional array in PyTorch which is capable of representing data of any dimension. They are also used for hardware acceleration and can be executed on parallel or graphics processing units which speeds up matrix calculations that are central to deep learning.
For ex: A 10000×10000 *matrix multiplication can be approximately 30 times faster on a GPU.*

9.4 The problem with backward pass and its solution

To calculate backward pass, we need to compute the derivative of loss wrt every parameter which is manually impossible for a deep neural network. PyTorch's automatic differentiation engine `torch.autograd` solves this problem by automatically computing the gradients.

The idea: PyTorch builds a Directed Acyclic Graph (DAG) during forward pass that records all operations and tensors.

Code Snippet:

```
# Tensors w and b have requires_grad=True
z = torch.matmul(x, w) + b
loss = loss_fn(z, y)

# The magic happens here!
loss.backward()

# Gradients are now available
print(w.grad)
print(b.grad)
```

`requires_grad='True'` is used to start the gradient engine and `torch.no_grad()` is used to pause it.

9.5 Assignment Insights

In this assignment, we manually implemented a Multi-Layer Perceptron to map a non-linear "Radiation Ring," by navigating the complexities of binary classification using raw tensor operations and manual Stochastic Gradient Descent. This process uses rigorous application of the chain rule through PyTorch's autograd engine while managing critical constraints such as numerical stability in loss calculations and the effective generalization of data despite inherent noise.